

ME – 504

Project – Question 6a

Approach-1: Predict the maximum Von Mises stress for the given volume fraction distribution using CNN.

Approach-2: Predict the Von Mises stress field for a given volume fraction distribution using UNet. Use this approach to finally predict the maximum Von Mises stress.

Which approach is superior, compare the pros and cons. For example, compare the data requirement of the approach-1 vs approach-2.

Approach-1: Predict the maximum Von Mises stress for the given volume fraction distribution using CNN.

- A Convolutional Neural Network (CNN) is designed to automatically detect patterns and spatial relationships in data using convolution filters. It is commonly used for image processing, but can also be applied to structured engineering data by reshaping inputs into grid-like formats. In this project, the CNN learns the relationship between strut thickness distributions and the resulting maximum Von Mises stress values from FEA simulations.
- The model is given an output.xlsx file with each row signifying a sample and each row thickness depending on 226 thickness parameters. For each sample, we're mapping the stress values over 3305 values.
- The approach here is to find the max value of stress in each row which is then used as training data to predict the max stress.

- We've reshaped the data from a 1-dimensional array to a square of side 16 with padding just for simplicity.
- The CNN model follows the sequence: Convolution layer - Max Pooling Layer - Convolution layer - Max Pooling Layer - Flatten - Dense - Dropout
- Early stopping is also added to help with regularisation.

```
model = Sequential()
model.add(Conv2D(32,(3,3),
                activation='relu',
                input_shape=(16,16,1)))

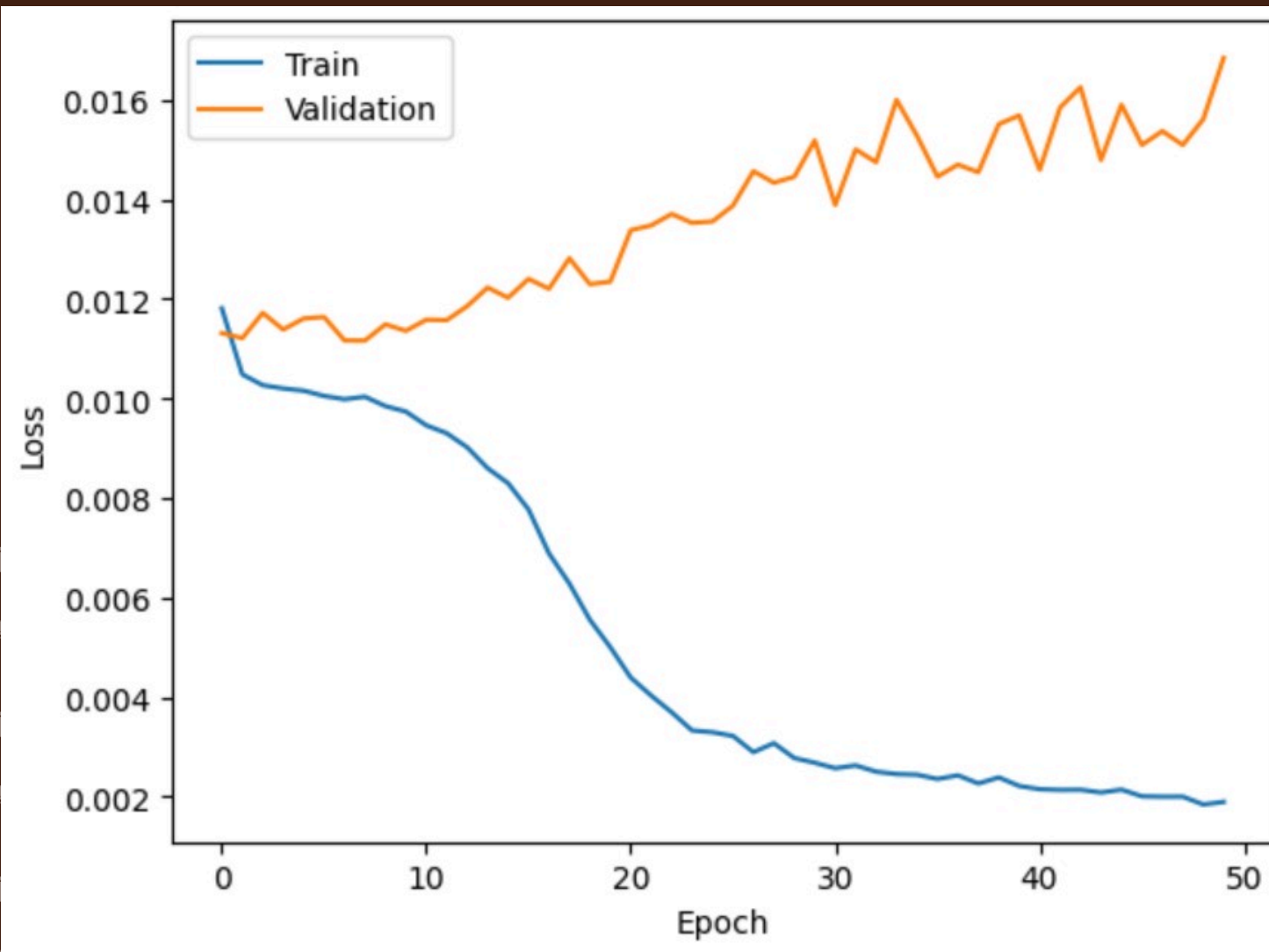
'local/lib/python3.12/dist-packages/keras/src/la
er().__init__(activity_regularizer=activity_reg
```

```
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(64,(3,3),
                activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())

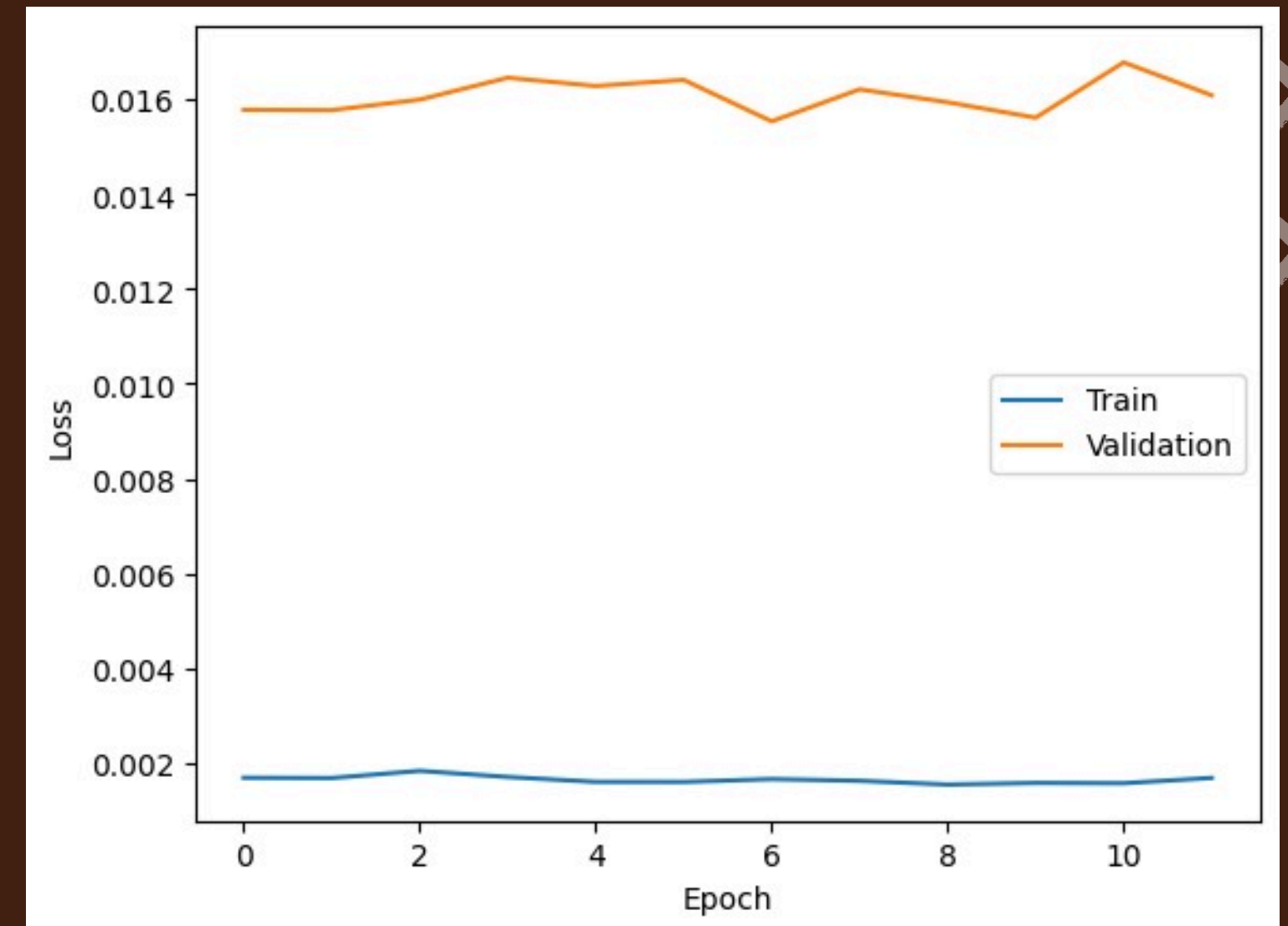
model.add(Dense(128,
                activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1))
```

1. Training vs. testing error on 1st run



- This model uses 50 epochs and 320 parameters.
- This is clearly a sign of overfitting since the training data is very low and the validation data increases
- To deal with we add EarlyStopping

2. Training vs. testing error after EarlyStopping

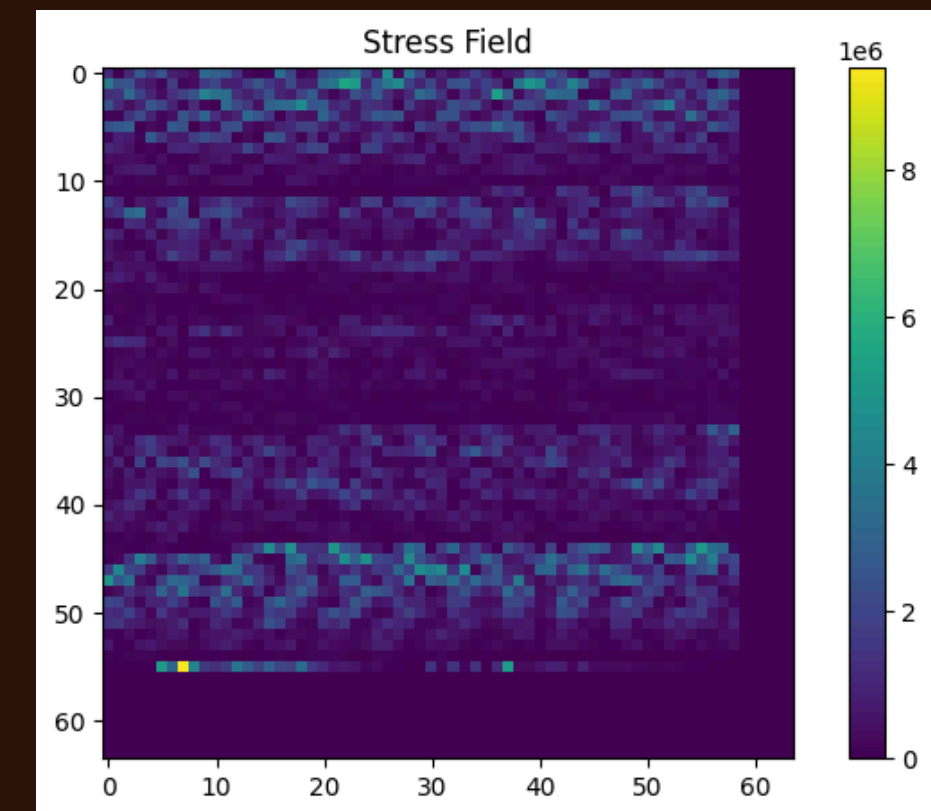
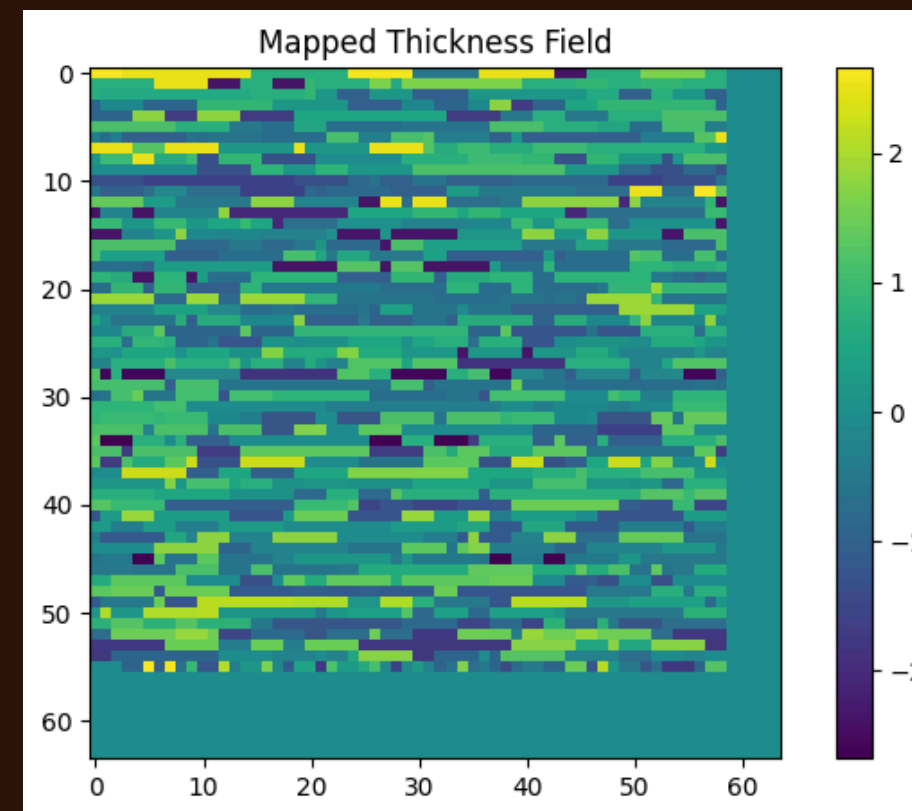


- Even though the epochs stop after 10. We still see gap between the validation and the training data.
- This shows a drawback of CNN since it's directly uses max stresses instead of considering the spatial variation of the values.

Predicted Max Stress:
18509726.0
Actual Max Stress:
9094219.75618857

Approach 2 – Using U-Net to predict a field of Von-Misses Stresses

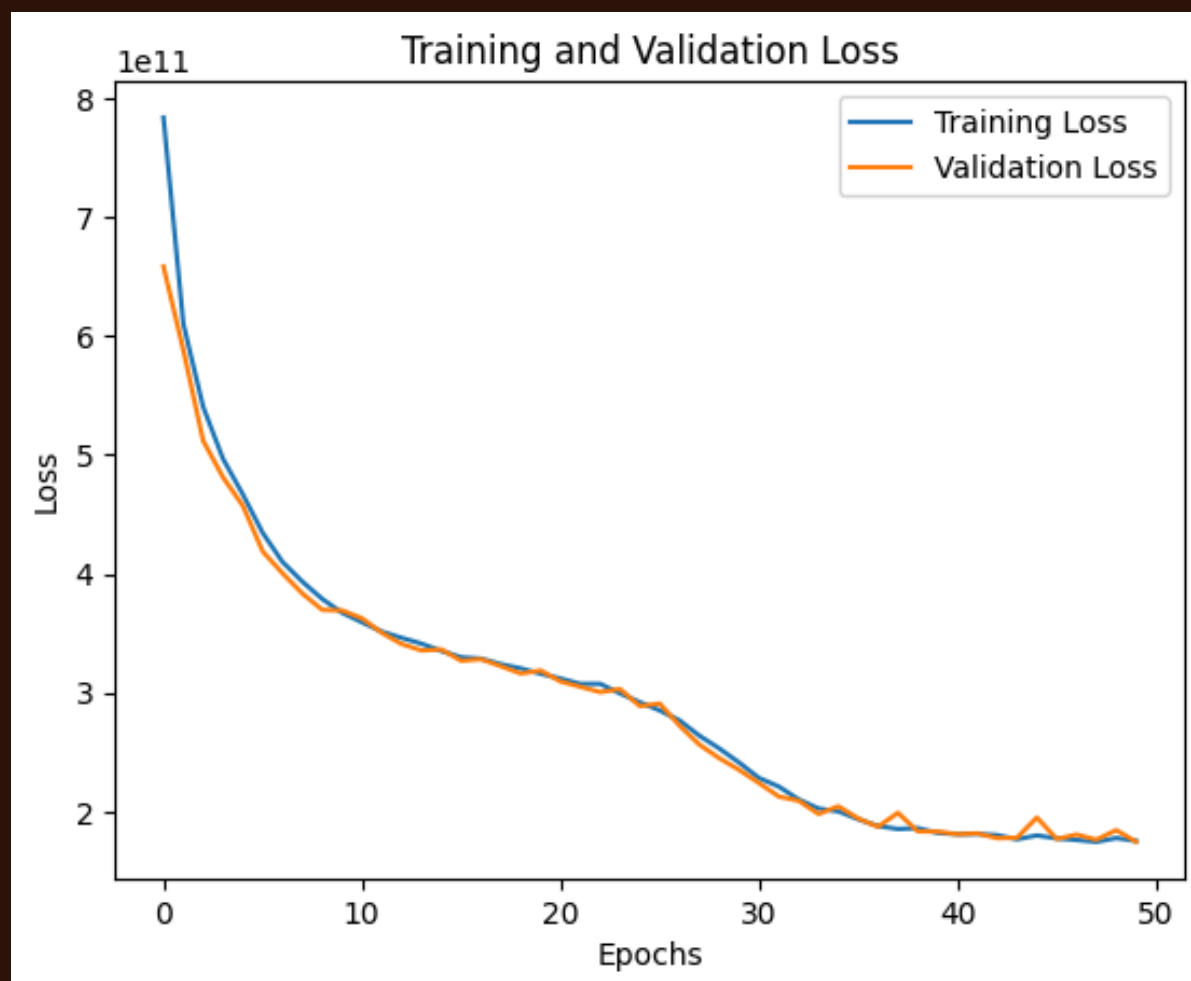
- **U-Net** is a **specialised CNN** that gives a **spatial/coordinate based output**. While in CNN we got the average maximum stress, in U-Net we get the value of maximum stress at each node.
 - Since at the end we are finding the maximum value of stress for each individual node, and analysing the global stress field as well, an **encoder → decoder format** is used in U-Net, with the basic framework of CNN.
 - U-Net uses Functional APIs because it branches and merges as compared to CNN which uses Sequential forms by simply stacking the layers on top of another.
-
- Data was extracted from the thickness and stress files.
 - Thickness depends on only 226 parameters
 - There are 3304 coordinates, so 3304 values of stress at each node for each strut.
 - The 3304 coordinates were plotted to a grid of 56 x 59 as a reference
 - The thickness values were approximated to a 14 x 17 grid to plot the 226 values.
 - This 14 x 17 grid was then mapped to the 56 x 59 grid.
 - A padding layer was applied to make the final images 64 x 64



OVERALL MODEL:

- We want to predict a grid of Stress values for thickness grid(input - 64 x 64)
- Apply 2D convolutional layer to learn spatial features such as edges, local thickness gradients, local concentrations etc. from the map
- Followed by max pooling layer
- Format followed: Encoder → Bottleneck → Decoder

- For around 4000 trainable parameters, 50 epochs, and batch size 32, the following outcomes were observed:



- Graph shows that model is in underfitting region.
 - Capacity must be increased to get out of underfitting
- Solution: no. of trainable parameters were increased to 1,861,697

```
32/32 ————— 21s 638ms/step - loss: 174006321152.0000 - mae: 231891.2188  
Test Loss: 174006321152.0  
Test MAE: 231891.21875
```

- Test loss is extremely high
- Input data was normalised before next run

FINAL CONCLUSION ON CNN VS. U-NET

- **The** goal of this project was to compare how these 2 methods work on predicting the max stress values.
- As we can see in the CNN case, the training and validation data follow the same curve but there is a constant difference between them. This indicates that the model is not able to entirely predict the values correctly and it keeps a bias which it can't get rid off.
- This shows us the limitation of CNN since it doesn't consider spatial relation between the points and we've just considered the maximum value of stress from every sample without considering how they vary across the sample.
- In such a case, it better to use U-net since it considers the stress values from each point and works on both encoding and decoding the data, giving us the stress values over the entire area.
- This helps in a much better prediction of the maximum stress values.